EECS3311 Software Design

Fall 2026

Professor: Song Wang

Course Project: Design, Build, and Test an AI Agent

Orion Threat Intelligence

Stage 2 Report

Name: Saransh Mahajan

Student ID: 221563945

Table of Contents

1. Project Overview 4
1.1 Target Users: 4
1.2 AI and LLM Integration 4
1.3 Current Working Functionality 4
1.4 Limitations and Incomplete Functionality 5
2. Feature to Implementation Mapping 5
F01 - Ingest Threat Advisories 5
F02 - Automated Threat Bulletin Summarization 6
F03 - Natural Language Threat Landscape Querying (AI Chat Bot) 7
F04 - Insider Risk Log Correlation 8
F05 - Automated Mitigation Strategy Generation & Execution 8
F06 - Structured Indicator of Compromise (IoC) Extraction 9
F07 - Executive Threat Intelligence Report Export 10
F08 - Historical Threat Repository Search 11
F09 - Dynamic Advisory Triage & Priority Scoring 11
F10 - Security Control Gap Analysis 12
F11 - Multi-Select Batch Export 12
F12 - Live Threat News Carousel 13
3. Implemented Design Patterns 14
3.1 Observer Pattern 14
3.2 Strategy Pattern 14
3.3 Facade Pattern 15
3.4 Command Pattern 16
3.5 Factory Method Pattern 16
4. Changes and Rationale 17
4.1. LLM Backend Provider (Groq API Migration) 17
4.2. Multi-Select Batch Export (F07 Extension) 17
4.3. Portable Execution & Reproducibility Assets 18
4.4 Live Threat News Carousel 18
5. AI Tools and Collaboration Log 19
5.1. Google Gemini (Personal AI Collaborator) 19
5.2. Groq Cloud API (Runtime AI Engine) 19

6. Development and Verification Summary 20
6.1. Structured Commit Summary 20
6.2 Checks Performed to Verify Implementation 25
6.3 Important Defects Found and Corrected 25
6.4 Remaining Limitations 26
7. Demonstration Index 26

1. Project Overview

Orion Threat Intelligence (CTI) is a dual-mode, enterprise-grade Java and JavaFX application and AI agent framework designed to combat alert fatigue and data fragmentation in Security Operations Centers (SOCs). Traditional security tools excel at deterministic database queries and API ingestion, but struggle with synthesizing lengthy unstructured threat disclosures and contextualizing them against specific enterprise infrastructure. Orion bridges this gap by combining deterministic core software (such as SQLite caching, regex parsing, and command execution stacks) with non-deterministic AI agent reasoning powered by the Groq LPU inference engine.

1.1 Target Users:

1. **SOC Analysts:** Personnel who need rapid summarization of dense vulnerability disclosures and automated extraction of actionable Indicators of Compromise (IoCs).
2. **Threat Intelligence Specialists:** Analysts tracking emerging adversary Tactics, Techniques, and Procedures (TTPs) across external feeds.
3. **Security Supervisors:** Leaders requiring automated risk triage, security control gap analyses, and exportable executive threat briefings.

1.2 AI and LLM Integration

The AI integration is architecturally isolated from direct system execution to ensure safety and maintain a human-in-the-loop validation barrier.

- **Backend Provider:** Groq Cloud API routing to the high-performance **openai/gpt-oss-120b** model variant.
- **Orchestration:** Managed centrally within the ‘llm’ and ‘controller’ packages. The deterministic software packages parse incoming data, and ‘PromptBuilder’ structures constraints before dispatching asynchronous requests via ‘GroqLLMClient’.
- **Scope of AI Tasks:** Automated vulnerability summarization, enterprise remediation playbook generation, security control gap analyses, automated triage priority scoring, and natural language threat landscape querying.

1.3 Current Working Functionality

The implemented system features a fully operational, production-ready codebase supporting:

- **Dual-Mode Interfaces:** A modern glassmorphism JavaFX graphical workspace (OrionGUI) and an interactive, ANSI-colored terminal shell (OrionCLI).

- **Feed Synchronization & Persistence:** Real-time ingestion and local SQLite caching of threat advisories from the CISA Known Exploited Vulnerabilities (KEV) catalog and NIST National Vulnerability Database (NVD).
- **Security Tooling:** Automated audit log correlation and regex-based extraction of malicious indicators of compromise (IPs, hashes, domains) from unstructured text.
- **Batch Operations & Reporting:** Multi-select CVE export functionality allowing analysts to bundle multiple vulnerability records into structured Markdown briefing reports.
- **Command Auditing:** A Command Pattern implementation tracking infrastructure modifications on a history stack with full execution and undo capabilities.

1.4 Limitations and Incomplete Functionality

To maintain complete academic transparency, the current implementation acknowledges the following limitations:

- **Cloud API Rate Limits:** Heavy, concurrent bulk evaluations depend on Groq Cloud API rate limits; excessive automated requests can trigger throttling exceptions, requiring graceful fallback mechanisms to base CVSS scores.
- **Log Format Constraints:** While the log correlation engine successfully processes standardized audit logs (such as the provided ‘log.example.txt’), highly unstructured or non-standard custom SIEM export formats require strict adherence to expected headers for optimal anomaly detection.

2. Feature to Implementation Mapping

F01 - Ingest Threat Advisories

- **Stage 1 feature/reference:** F01 - External Threat Advisory Ingestion
- **Status:** Implemented
- **Implemented classes/files and responsibilities:**
  - OrionGUI / OrionCLI: Presentation layers capturing the user's synchronization trigger.
  - AgentController: Central deterministic orchestrator coordinating feed retrieval and persistence.

  - AdvisoryParserFactory: Factory class responsible for instantiating the correct schema parser based on feed source.
  - NvdJsonParser: Concrete parser processing raw NIST NVD and CISA KEV JSON payloads into domain objects.
  - AdvisoryRepository: Data access layer managing database transactions and local SQLite persistence.
- **Key methods:** syncFeeds(), createParser(), parse(), saveAll()
- **Design patterns:** Factory Method Pattern
- **Video timestamp link:** https://www.youtube.com/watch?v=P0hejUG7d7c&t=33s

Collaboration & Execution Flow: The user initiates a synchronization request via the GUI button or CLI command. The presentation layer invokes AgentController.syncFeeds(), which requests an appropriate parser from AdvisoryParserFactory. The factory returns NvdJsonParser, which issues an HTTP request, parses the incoming JSON payload into ThreatAdvisory domain objects, and passes them to AdvisoryRepository.saveAll() to store them in the local SQLite database. The UI is then updated with an ingestion summary. This feature is **Deterministic**.

F02 - Automated Threat Bulletin Summarization

- **Stage 1 feature/reference:** F02 - Automated Threat Bulletin Summarization
- **Status:** Implemented
- **Implemented classes/files and responsibilities:**
  - OrionGUI: Renders the generated summary card in rich HTML/Markdown.
  - AgentController: Routes the summary request from the UI to the AI subsystem.
  - AgentFacade: Orchestrates the underlying reasoning components without exposing subsystem complexities.
  - PromptBuilder: Formats threat metrics and raw descriptions into structured constraint prompts.
  - GroqLLMClient: Communicates asynchronously with the Groq Cloud API endpoint (openai/gpt-oss-120b).
- **Key methods:** handleNaturalQuery(), processGoal(), buildQueryPrompt(), generate()

- **Design patterns:** Facade Pattern, Observer Pattern
- **Video timestamp link:** https://www.youtube.com/watch?v=P0hejUG7d7c&t=65s

Collaboration & Execution Flow: The analyst selects a vulnerability record and requests an executive summary. The GUI passes the request to AgentController, which delegates execution to AgentFacade.processGoal(). The facade utilizes PromptBuilder.buildQueryPrompt() to construct a structured prompt, which is dispatched asynchronously via GroqLLMClient.generate(). The LLM returns a markdown-formatted summary, which is sent back through the facade and controller to update the UI asynchronously via the observer mechanism. This feature is **AI-based**.

F03 - Natural Language Threat Landscape Querying (AI Chat Bot)

- **Stage 1 feature/reference:** F03 - Natural Language Threat Landscape Querying
- **Status:** Implemented
- **Implemented classes/files and responsibilities:**
    - OrionGUI / OrionCLI: Captures conversational queries from the user input field or terminal prompt.
    - AgentController: Routes ad-hoc questions to the intelligence subsystem.
    - AgentFacade: Manages high-level goal processing and contextual coordination.
    - AgentMemoryManager: Maintains a session conversationBuffer to track conversational history and short-term memory.
    - GroqLLMClient: Interfaces with the Groq inference engine to generate grounded conversational responses.
- **Key methods:** handleNaturalQuery(), processGoal(), retrieveContext(), appendContext(), generate()
- **Design patterns:** Facade Pattern
- **Video timestamp link:** https://www.youtube.com/watch?v=P0hejUG7d7c&t=87s

Collaboration & Execution Flow: The user inputs an ad-hoc security question into the chat interface. AgentController calls AgentFacade.processGoal(), which queries AgentMemoryManager.retrieveContext() to pull previous conversational turns. The prompt and memory stack are sent to GroqLLMClient. The synthesized response is saved back to memory via AgentMemoryManager.appendContext() and surfaced in the

UI workspace. This feature is **Hybrid** (deterministic local retrieval combined with AI synthesis).

F04 - Insider Risk Log Correlation

- **Stage 1 feature/reference:** F04 - Insider Risk Log Correlation
- **Status:** Implemented
- **Implemented classes/files and responsibilities:**
  - OrionGUI: Handles audit log file selection and upload.
  - AgentController: Directs the parsing workflow and risk evaluation.
  - ToolManager: Selects and invokes the appropriate utility tool from its registry.
  - LogParserTool: Deterministically parses CSV/syslog files to extract structural anomalies and access spikes.
  - AgentFacade: Bridges extracted anomalies with external threat vectors.
  - GroqLLMClient: Evaluates behavioral overlaps against known attacker TTPs.
  - RiskLogRepository: Persists correlated risk alerts to local storage.
- **Key methods:** correlateLogs(), executeTool(), execute(), evaluateTriage(), generate()
- **Design patterns:** Strategy Pattern, Observer Pattern
- **Video timestamp link:** https://www.youtube.com/watch?v=P0hejUG7d7c&t=121s

Collaboration & Execution Flow: The analyst uploads an access log file through OrionGUI. AgentController uses ToolManager to invoke LogParserTool.execute(), which deterministically extracts statistical outliers. The controller passes these anomalies to AgentFacade, which queries GroqLLMClient to evaluate behavioral overlaps. Structured risk alerts are saved via RiskLogRepository and broadcasted asynchronously to the UI via the SecurityObserver pattern. This feature is **Hybrid**.

F05 - Automated Mitigation Strategy Generation & Execution

- **Stage 1 feature/reference:** F05 - Automated Mitigation Strategy Generation
- **Status:** Implemented

- **Implemented classes/files and responsibilities:**
    - OrionGUI: Displays the generated playbook and captures human-in-the-loop approval.
    - AgentController: Coordinates mitigation planning and command execution phases.
    - AgentFacade: Requests structured remediation plans from the LLM.
    - GroqLLMClient: Outputs JSON-formatted mitigation playbooks.
    - CommandManager: Maintains a history stack to queue, track, execute, or undo system modifications.
    - BlockIpCommand: Concrete command encapsulating specific network rule modifications (execute() / undo()).
- **Key methods:** generateMitigation(), planMitigation(), executeCommand(), invoke(), execute()
- **Design patterns:** Command Pattern, Facade Pattern
- **Video timestamp link:** https://www.youtube.com/watch?v=P0hejUG7d7c&t=169s

Collaboration & Execution Flow: The analyst requests a mitigation plan for a specific threat. AgentFacade queries GroqLLMClient to generate a remediation playbook, which the system parses into concrete command objects like BlockIpCommand. Upon human approval in the GUI, AgentController.executeCommand() passes the action to CommandManager.invoke(), which calls BlockIpCommand.execute() to apply the rule and pushes it onto the history stack for auditing or rollback. This feature is **AI-based** (with deterministic command execution).

F06 - Structured Indicator of Compromise (IoC) Extraction

- **Stage 1 feature/reference:** F06 - Structured Indicator of Compromise (IoC) Extraction
- **Status:** Implemented
- **Implemented classes/files and responsibilities:**
    - OrionGUI: Captures raw unstructured threat text pasted by the user.
    - AgentController: Forwards text to the intelligence subsystem.
    - AgentFacade: Directs the extraction and normalization workflow.

    - AgentPlanner: Determines necessary extraction and parsing steps.
    - GroqLLMClient: De-fangs and normalizes complex indicators (IPs, hashes, domains).
- **Key methods:** handleNaturalQuery(), processGoal(), createPlan(), generate()
- **Design patterns:** Facade Pattern
- **Video timestamp link:** https://www.youtube.com/watch?v=P0hejUG7d7c&t=208s

Collaboration & Execution Flow: The analyst pastes unstructured threat intelligence text into the extraction panel. The system applies deterministic regex patterns to catch well-formed indicators, while complex or obfuscated segments are passed via AgentFacade and AgentPlanner to GroqLLMClient with a strict JSON schema prompt for normalization. The deduplicated results are rendered in a structured table. This feature is **Hybrid**.

F07 - Executive Threat Intelligence Report Export

- **Stage 1 feature/reference:** F07 - Executive Threat Intelligence Report Export
- **Status:** Implemented + Modified (with enhanced multi-select batch export)
- **Implemented classes/files and responsibilities:**
    - OrionGUI: Initiates the file save dialog and captures record selections.
    - AgentController: Orchestrates data retrieval and file writing operations.
    - AdvisoryRepository: Fetches single or multiple database records based on user selection.
- **Key methods:** searchAdvisories(), findById()
- **Design patterns:** Repository Pattern / Standard Data Access
- **Video timestamp link:** https://www.youtube.com/watch?v=P0hejUG7d7c&t=258s

Collaboration & Execution Flow: The user selects one or more threat records in the GUI table (supporting both single selection and multi-select via Ctrl/Cmd + click) and clicks "Export Report". AgentController queries AdvisoryRepository using record identifiers to retrieve full threat details. The controller maps these entities into a clean Markdown template and executes standard Java I/O operations to save the briefing securely to the local disk. This feature is **Deterministic**.

F08 - Historical Threat Repository Search

- **Stage 1 feature/reference:** F08 - Historical Threat Repository Search
- **Status:** Implemented
- **Implemented classes/files and responsibilities:**
    - OrionGUI / OrionCLI: Captures search filters (date range, severity, vendor).
    - AgentController: Translates UI parameters into repository queries.
    - AdvisoryRepository: Executes parameterized SQL statements against the local SQLite database.
- **Key methods:** searchAdvisories(), findByCriteria()
- **Design patterns:** Repository Pattern
- **Video timestamp link:** https://www.youtube.com/watch?v=P0hejUG7d7c&t=290s&pp=0gcJCWMAwfN6Pr3D

Collaboration & Execution Flow: The analyst configures filter thresholds (e.g., minimum CVSS score) in the search panel. The presentation layer invokes AgentController.searchAdvisories(), which bypasses the AI layer entirely and calls AdvisoryRepository.findByCriteria(). This executes an optimized SQL query against the local database, returning matching records to populate the UI table instantly without consuming LLM API tokens. This feature is **Deterministic**.

F09 - Dynamic Advisory Triage & Priority Scoring

- **Stage 1 feature/reference:** F09 - Dynamic Advisory Triage & Priority Scoring
- **Status:** Implemented
- **Implemented classes/files and responsibilities:**
    - OrionGUI: Renders calculated triage priority badges and justification notes.
    - AgentController: Initiates scoring requests for untriaged advisories.
    - AgentFacade: Manages qualitative threat evaluation logic.
    - GroqLLMClient: Evaluates threat exploitability against local environment profiles.

- **Key methods:** generateMitigation(), evaluateTriage(), generate()
- **Design patterns:** Facade Pattern
- **Video timestamp link:** https://www.youtube.com/watch?v=P0hejUG7d7c&t=322s

Collaboration & Execution Flow: When new advisories are ingested or selected for triage, AgentController calls AgentFacade.evaluateTriage(). The facade instructs GroqLLMClient to evaluate the qualitative threat description against the enterprise tech stack, returning a customized severity score and rationale that overrides generic base CVSS ratings. If rate limits are reached, the system gracefully falls back to deterministic CVSS base thresholds. This feature is **AI-based**.

F10 - Security Control Gap Analysis

- **Stage 1 feature/reference:** F10 - Security Control Gap Analysis
- **Status:** Implemented
- **Implemented classes/files and responsibilities:**
  - OrionGUI: Triggers gap analysis upon user request.
  - AgentController: Manages data collection between local security baselines and threat records.
  - AgentFacade: Bridges control inventories with the AI evaluation engine.
  - GroqLLMClient: Performs cross-reference matrix analysis to identify unmitigated attack vectors.
- **Key methods:** handleNaturalQuery(), evaluateGaps(), generate()
- **Design patterns:** Facade Pattern
- **Video timestamp link:** https://www.youtube.com/watch?v=P0hejUG7d7c&t=358s

Collaboration & Execution Flow: The analyst requests a gap analysis for a specific vulnerability. AgentController retrieves active baseline security controls and passes them alongside the threat profile to AgentFacade.evaluateGaps(). GroqLLMClient evaluates coverage overlaps to identify missing defensive controls, returning a structured gap matrix to the presentation layer. This feature is **AI-based**.

F11 - Multi-Select Batch Export

- **Stage 1 feature/reference:** New Feature (Extension of F07)

- **Status:** Implemented
- **Implemented classes/files and responsibilities:**
    - OrionGUI: Extended table selection model enabling multi-row selection (Ctrl/Cmd + click).
    - AgentController: Iterates over selected identifiers to batch-fetch records and compile a consolidated markdown report.
- **Key methods:** searchAdvisories(), exportBatch()
- **Design patterns:** Repository Pattern
- **Video timestamp link:** https://www.youtube.com/watch?v=P0hejUG7d7c&t=384s

Collaboration & Execution Flow: The security analyst holds Ctrl or Cmd while selecting multiple threat rows in the GUI data table and clicks "Export Selected". AgentController aggregates the selected CVE IDs, queries AdvisoryRepository for each record, compiles them into a unified multi-advisory briefing format, and writes the output file locally. This feature is **Deterministic**.

F12 - Live Threat News Carousel

- **Stage 1 feature/reference:** New Feature (UI Enhancements)
- **Status:** Implemented
- **Implemented classes/files and responsibilities:**
    - OrionGUI: Renders the bottom footer container with an animated carousel effect.
    - NewsFeedClient / RSS Parser: Fetches real-time cybersecurity news headlines and summaries asynchronously.
    - AgentController: Coordinates background fetching of RSS/API news payloads.
- **Key methods:** loadNewsTicker(), startCarouselAnimation()
- **Design patterns:** Observer Pattern (for asynchronous ticker updates)
- **Video timestamp link:** https://www.youtube.com/watch?v=P0hejUG7d7c&t=418s
- **Collaboration & Execution Flow**: Upon application startup, AgentController triggers a background task to fetch the latest cybersecurity headlines. Once retrieved, the data is pushed asynchronously via the observer mechanism to OrionGUI, which populates the bottom footer carousel and rotates headlines smoothly across the interface. This feature is **Deterministic**.

3. Implemented Design Patterns

3.1 Observer Pattern

- **Design Problem Addressed:** Handles asynchronous, long-running background operations—such as threat feed synchronization, log correlation, and LLM inference—without freezing or blocking the JavaFX presentation layer.
- **Participating Classes & Roles:**
    - SecurityObserver (Interface): Establishes the asynchronous notification contract (onSecurityEvent).
    - OrionGUI (Concrete Observer): Implements the interface to update visual components and data tables asynchronously on the JavaFX application thread.
    - AgentController (Subject): Broadcasts state changes and asynchronous events as background tasks complete.
- **Relevant Source Files & Methods:**
    - ca.yorku.eecs3311.orion.observer.SecurityObserver.java (onSecurityEvent)
    - ca.yorku.eecs3311.orion.boundary.OrionGUI.java (onSecurityEvent)
    - ca.yorku.eecs3311.orion.controller.AgentController.java
- **Implementation & Utility:** Enforces a strict separation of concerns, decoupling domain logic from the graphical user interface. If omitted, running the system in a headless state (OrionCLI) would be difficult because the controller would require hardcoded references to JavaFX components.
- **Stage 1 Comparison: Matches Stage 1.** No architectural changes were made to this pattern.

3.2 Strategy Pattern

- **Design Problem Addressed:** Solves the problem of executing varied data retrieval and parsing algorithms dynamically at runtime based on agent planning or user selection.
- **Participating Classes & Roles:**
    - Tool (Strategy Interface): Defines a universal execution contract (execute).

  - NvdAdvisoryTool, LogParserTool, AdvisoryRepoSearchTool (Concrete Strategies): Encapsulate unique, specialized logic required for specific system tasks.
  - ToolManager (Context / Invoker): Maintains a registry of tools to dispatch execution requests dynamically.
- **Relevant Source Files & Methods:**
  - ca.yorku.eecs3311.orion.tool.Tool.java (execute)
  - ca.yorku.eecs3311.orion.tool.LogParserTool.java, NvdAdvisoryTool.java
  - ca.yorku.eecs3311.orion.tool.ToolManager.java (registerTool, executeTool)
- **Implementation & Utility:** Adheres strictly to the Open/Closed Principle; new integration capabilities can be added by creating new tools without altering core agent logic or writing brittle conditional blocks.
- **Stage 1 Comparison: Matches Stage 1.** Maintained identically to the initial design specification.

3.3 Facade Pattern

- **Design Problem Addressed:** Orchestrates a complex, multi-step AI reasoning subsystem—encompassing prompt building, memory management, tool coordination, and API communication—without exposing internal intricacies to the UI boundary layer.
- **Participating Classes & Roles:**
  - AgentFacade (Facade): Exposes simplified, high-level entry points (processGoal, evaluateTriage, evaluateGaps).
  - AgentPlanner, ToolManager, AgentMemoryManager, LLMClient (Subsystem Components): Handle granular tasks like contextual memory retrieval, prompt construction, and API communication.
- **Relevant Source Files & Methods:**
  - ca.yorku.eecs3311.orion.controller.AgentFacade.java (processGoal, evaluateTriage)
  - ca.yorku.eecs3311.orion.llm.AgentMemoryManager.java
- **Implementation & Utility:** Establishes a clean, stable entry point for the presentation layer to interact with non-deterministic agent capabilities, preventing GUI and CLI controllers from duplicating complex orchestration logic.

- **Stage 1 Comparison: Matches Stage 1.** Retained as the core boundary-to-agent bridge.

3.4 Command Pattern

- **Design Problem Addressed:** Addresses the critical security requirement of executing AI-generated defensive remediations safely, ensuring actions can be tracked, queued, audited, and rolled back.
- **Participating Classes & Roles:**
  - SecurityCommand (Interface): Mandates execute(), undo(), and getDescription() contracts.
  - BlockIpCommand (Concrete Command): Encapsulates specific infrastructure state modifications (e.g., target IP addresses).
  - CommandManager (Invoker): Invokes actions while pushing them onto an auditable history stack.
- **Relevant Source Files & Methods:**
  - ca.yorku.eecs3311.orion.command.SecurityCommand.java
  - ca.yorku.eecs3311.orion.command.BlockIpCommand.java (execute, undo)
  - ca.yorku.eecs3311.orion.command.CommandManager.java (invoke, undoLast)
- **Implementation & Utility:** Treats infrastructure modifications as discrete objects, allowing security administrators to enforce human-in-the-loop validation before automated agents apply network rules, with full automated rollback support.
- **Stage 1 Comparison: Matches Stage 1.** Fully implemented according to design specifications.

3.5 Factory Method Pattern

- **Design Problem Addressed:** Solves the problem of instantiating the correct deserialization algorithms for heterogeneous external threat feeds utilizing varying schemas (such as JSON from CISA KEV and NIST NVD catalogs).
- **Participating Classes & Roles:**
  - AdvisoryParserFactory (Creator): Exposes a createParser() method evaluating format strings to instantiate parsers.
  - AdvisoryParser (Product Interface): Defines standard parsing contracts.

    - NvdJsonParser (Concrete Product): Executes schema-specific parsing logic.
- **Relevant Source Files & Methods:**
    - ca.yorku.eecs3311.orion.api.AdvisoryParserFactory.java (createParser)
    - ca.yorku.eecs3311.orion.api.NvdJsonParser.java (parse)
- **Implementation & Utility:** Centralizes object creation, isolating feed ingestion tools from parser initialization complexities and making it effortless to integrate future vulnerability catalogs.
- **Stage 1 Comparison: Matches Stage 1.** Fully implemented as originally designed.

4. Changes and Rationale

4.1. LLM Backend Provider (Groq API Migration)

- **Original design:** Planned utilization of *Google Gemini 2.5 Flash* as the primary reasoning engine and *Groq LLaMA 3.3 (70B)* as a secondary fallback model via separate provider implementations (GeminiLLMClient and GroqLLMClient).
- **Implemented change:** Switched runtime inference routing exclusively to GroqLLMClient targeting the high-performance openai/gpt-oss-120b model endpoint via Groq's LPU inference engine, permanently removing the unused Gemini client code and its associated dependencies.
- **Reason:** Encountered persistent 503 service unavailability and rate-limiting errors with the Gemini endpoint during development and integration testing, necessitating a pivot to Groq's high-speed LPU infrastructure for operational stability.
- **Impact:** Zero structural impact on the feature or controller layer because the LLMClient interface abstraction successfully decoupled network transport specifics from the AgentFacade and PromptBuilder.

4.2. Multi-Select Batch Export (F07 Extension)

- **Original design:** Feature F07 (Export Structured Threat Briefing) was designed around single-item record selection and export workflows from the GUI threat table.

- **Implemented change:** Extended the GUI table selection model to support multi-row selection (Ctrl/Shift + click) combined with a batch export routine in AgentController.
- **Reason:** Security analysts frequently require consolidated intelligence reports spanning multiple related vulnerabilities rather than isolated single-CVE documents.
- **Impact:** Enhances analytical utility without altering core database access patterns, allowing security personnel to bundle and export multi-advisory briefing summaries seamlessly.

4.3. Portable Execution & Reproducibility Assets

- **Original design:** Stage 1 focused exclusively on core Java source packages and standard command-line compilation scripts without standardized environment packaging.
- **Implemented change:** Added automated Windows batch launchers (run_gui.bat, run_cli.bat) and a dedicated configuration/sample asset structure (.env.example, report.example.md, and log.example.txt).
- **Reason:** To ensure zero-friction reproducibility, enabling evaluators to build, configure, and test all GUI, CLI, log correlation, and IOC extraction features instantly.
- **Impact:** Streamlines grading and evaluation workflows by providing immediate out-of-the-box test assets without requiring manual path configurations or external mock data generation.

4.4 Live Threat News Carousel

- **Original design:** GUI layout focused strictly on the core threat table, AI console, and control panels without an integrated live news feed.
- **Implemented change:** Added a dynamic, scrolling news carousel footer at the bottom of the OrionGUI workspace displaying real-time cybersecurity news**.**
- **Reason:** To provide security analysts with immediate situational awareness and industry context directly inside the dashboard without needing external news browsers.
- **Impact:** Enhances the overall user experience and contextual depth of the workspace without affecting core database or agent reasoning logic.

5. AI Tools and Collaboration Log

5.1. Google Gemini (Personal AI Collaborator)

- **Tool/Platform:** Google Gemini (Google AI Pro Tier)
- **LLM Model Name:** Gemini Pro
- **Development Tasks & Features:**
    - Architectural design assistance for structuring modular Java packages.
    - Debugging JavaFX concurrency and event-handling threads.
    - Crafting and refining the custom .bat launchers and SQLite integration scripts.
    - Writing documentation, structuring the project README.md, and formatting report requirements.
- **Representative Interaction Example:**

  User: "How should I structure the SecurityObserver interface so that background threat feed synchronizations don't freeze the JavaFX GUI thread?"

  Gemini: "You should define an onSecurityEvent(String eventType, Object data) contract in your observer interface. Have your OrionGUI implement this interface and register it with the AgentController. When background threads complete fetching feed data, have the controller execute updates using Platform.runLater() to ensure thread-safe UI rendering."
- **Token & Financial Cost:**
    - ***Token Volume****:* Standard developer session tier (~150,000 contextual tokens processed across multi-turn refactoring sessions).
    - ***Monetary Cost****:* $6.79/month (Google AI Pro subscription).

5.2. Groq Cloud API (Runtime AI Engine)

- **Tool/Platform:** Groq Developer Console
- **LLM Model Name:** Llama 3 variant (openai/gpt-oss-120b) routed via Groq's LPU inference engine
- **Development Tasks & Features:**

- Integrated directly into the Orion backend to power real-time AI features: executive threat summaries, automated enterprise mitigation playbooks, environment-specific triage reports, and security control gap analyses.

- **Representative Interaction Example:**

  System Payload (PromptBuilder & GroqLLMClient): Ingests raw CVE attributes for CVE-2024-1234 alongside enterprise profile settings, dispatching a strict JSON-schema constraint prompt to Groq's endpoint to return a structured mitigation playbook.

  Groq API Response: Returns high-speed, low-latency JSON payload containing step-by-step remediation commands, subsequently parsed into executable BlockIpCommand objects.

- **Token & Financial Cost:**

  - ***Token Volume**:* Dependent on runtime API inference calls during development and functional evaluation.
  - ***Monetary Cost**:* $0.00 (Utilized Groq's developer free tier quota).

6. Development and Verification Summary

6.1. Structured Commit Summary

#	Date	Commit Message	Link
1	Nov 5	Initial commit: Set up Maven structure, dependencies, and secure .gitignore	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/6c34e0528f9cd6df5684f53996fd9279f3df509d
2		Readding Stage 1 report. Lost it due to Force push to Github.	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/e52cde6e27e6d887a223f8ccaefff587342ece5b
3		Implement ThreatAdvisory domain model with Jackson annotations.	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/4d933025d7bced886e1c99d6b5d8e843d7fc6801
4		Implement DatabaseManager with JML contracts	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/7ac98f20fe7c17024417330f273184c53e757311

		and SQLite table setup.	
5		Add NvdApiClient for fetching and parsing vulnerability data	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/08dd7f4a000c2b198e19f118b5eb65f059695b40
6		Align with UML by implementing AdvisoryParser, NvdJsonParser, AdvisoryFilter and AdvisoryRepository	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/91df22cb877ce48cb44927bca480f5b6c9d5100b
7		Refactor NvdApiClient to delegate parsing responsibility to NvdJsonParser.	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/42141e858f49d3dc5661a6a7af61d389ba194f3a
8		Add LLMClient interface and GeminiLLMClient implementation	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/1303040fb54cd1ac3da0ec8bb4371584480a307f
9		Implement AgentFacade and AgentMemoryManager to orchestrate AI reasoning	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/f49b509472d734eb772f8881cd8c85ccb185b3b3
10		Implement PromptBuilder and connect AI Facade to AgentController.	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/dafcf34b7f2f59b1d17e1a264ab31585043883b7
11		Implement Command Pattern to safely execute and track security remediations	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/8a25edfed15aaa0d56ee86e67674704cf07da770
12		Implement Strategy Pattern for ToolManager and LogParserTool	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/58e80a745183b8e34699df490f62064a518abfe7
13		Implement Observer Pattern to decouple presentation layer from background tasks	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/21ec3bf2b0b5e1ea1391bbd98b0c5dfb09daa091

14		Implement OrionCLI boundary layer and application entry point	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/86de568dbe630757c523aef80c1141bfe7855dbe
15		Implement live Gemini API Integration with Jackson JSON mapping	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/6303c5de42ebdc4b6c772972df63792bd903b56e
16		Implement OrionGUI boundary layer and add JavaFX Maven dependencies	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/4c4a142bd4283f997c3084e5551e0c275917c316
17		Implement active SQLite queries in AdvisoryRepository for Feature F08	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/4e81525fac0e6ca218fcbcb043511833000ff5e7
18		Implement ReportExporter utility for Feature F07 Executive Threat Briefings	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/4e81525fac0e6ca218fcbcb043511833000ff5e7
19		Implement IocExtractor utility for Feature F06 Structured IoC Extraction	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/9e04e204860d287ee638299d62f58c3e11545c3d
20		Implement Feature F02 Automated Threat Bulletin Summarization	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/900468b08f57fdbc5ef09328e944b921b1b56df2
21		Implement Feature F09 Dynamic Advisory Triage and Priority Scoring	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/0bc0e3eab829dff152cf77abe0b4d6420db3f9cc
22		Implement Feature F05 Automated Mitigation Strategy Generation	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/4dded6494f56bf74d97af7f62a01205421bdf90b
23		Implement Feature F10 Security	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/e493676ff1eaacc6aac334c4391b417d419c137f

		Control Gap Analysis	
24		Implement active log parsing in LogParserTool for Feature F04	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/a84494144f62406d7b2010e59c4449184f24d0e3
25		Implement JUnit unit test suite for AgentController, commands, and IoC extraction	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/44519a5969ae9c4aafa1bd50fdea1002108e4233
26		Add JUnit 5 Maven dependency for test suite execution	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/d024789795f84197bf80760f36d15f111026a9b6
27	Nov 6,	Wire AI feature buttons to GUI and enable multiple selection	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/8c620318d5760736b0c86a37c74da4f63d619808
28		Implemented Groq Client and API, successful Human-AI Interaction	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/743afa9955d004094ebfd84d58c0757fd44a74af
29		Fix DatabaseManager schema initialization to include 7-column metadata structure	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/6a098af9dfd0020f3d015f7bc4abf7d25ff80d46
30		Successfully wire up multi-source threat feed ingestion for CISA and NIST using AdvisoryParserFactory	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/de8ff580922bdf1d791f83c6cdc66277148e4e30
31		Implemented WebView rendering for enhanced GUI user experience	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/19077e19e7b4066218ebd30fa6a065709d7f89a6
32		Refactor Orion GUI: Implement WebView AI console, add published date	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/4960c6d75a09a8797d0fd02b3bcd2f4c972f4476

		column, and remove duplicate NIST parser	
33		GUI interface complete	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/462469412eaaf7c6f2d5e766cd97c7ed64d65e9a
34		Finalise CTI dashboard and GUI layout	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/524a7ba0b21535863dc6f5d3f25c6c8d98a1b199
35		CLI interface fully operational	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/7ad46adfec216a77ed95dd14fe81e9f175cfb2d7
36		Refactor Gemini Client and API, not used anymore. Environment runs on Groq.	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/9f36ff6b0c8d4bc0f13562c72e564cec505bebf0
37		Added Portable launchers for GUI and CLI and polished UI/CLI	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/d6191453c52d038ca9c5b73139804323deb2da71
38		Added comprehensive project README with architecture and setup guide	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/b9176452832ece979c0cd5ba491d2db7a6fd754f
39		Updated AI & Human Collaborative Development seciton in README	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/66be8af63ca5760f1bd1d6362e371d398ff13351
40	Nov 7,	Added example files and example usage of GUI & CLI	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/3baeb7c318a4c8bc4d6d81a9e1c52a18e9407600
41		Modified Readme	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/1e5ef3d22932501c12314f9604de30ff3742d01d
42		Refactor OrionGUI: Add text-wrapping and break-all rules to chat bubbles and	https://github.com/saranmhjn/EECS3311-Orion-Threat-Intelligence/commit/e39943a7c79e39b5bdad5b3f2b7219569ce72c47

		WebView renderer to prevent text overflow	

6.2 Checks Performed to Verify Implementation

To validate system stability and feature correctness across both interface modes, the following verification checks were performed:

1. **Compilation & Dependency Resolution:** Executed clean Maven builds (mvn clean compile) across Windows environments to verify that all JavaFX and Groq SDK dependencies resolve correctly without compilation warnings.
2. **Launch & Boot Verification:** Tested zero-friction startup using both manual Maven commands and the portable batch scripts (run_gui.bat and run_cli.bat) for both GUI and CLI modes.
3. **Feed Ingestion & Database Persistence:** Triggered live CISA/NIST feed synchronizations, verifying that records are successfully fetched, parsed via the factory, and stored/queryable within the local SQLite database.
4. **AI Inference & Tool Execution:** Tested Groq-powered summary generation, automated triage scoring, and natural language queries, confirming valid JSON responses and correct prompt structuring.
5. **Security Command Auditing:** Verified that executing remediation commands correctly pushes actions onto the CommandManager history stack and that the undo() method successfully reverts infrastructure states.
6. **Log Correlation & IOC Extraction:** Ran tests against the provided sample_audit.log file and unstructured threat text blocks to validate deterministic regex matching and behavioral overlap reporting.

6.3 Important Defects Found and Corrected

- **Defect 1: JavaFX UI Thread Freezing During Feed Sync & LLM Inference**
    - *Issue:* Initial implementations of syncFeeds() and AI generation calls executed directly on the JavaFX application thread, causing the graphical interface to freeze during network latency spikes.
    - *Correction:* Implemented the SecurityObserver pattern and wrapped long-running operations in background worker threads, utilizing Platform.runLater() to safely broadcast state changes and update UI tables asynchronously.

- **Defect 2: Groq API Rate-Limiting & JSON Deserialization Faults**
    - *Issue:* Rapid multi-item triage requests occasionally triggered Groq API rate limits (HTTP 429) or returned markdown-wrapped JSON blocks that broke deterministic parsers.
    - *Correction:* Added strict JSON-schema prompt constraints within PromptBuilder and engineered a graceful fallback mechanism where the system defaults to deterministic CVSS base scoring if an LLM throttling exception occurs.
- **Defect 3: Cross-Platform Execution Discrepancies**
    - *Issue:* Hardcoded file paths in early CLI scripts caused execution failures when launched from different root directories.
    - *Correction:* Refactored the execution scripts (run_gui.bat and run_cli.bat) to utilize dynamic relative path resolution and automated Maven lifecycle hooks.

6.4 Remaining Limitations

- **Cloud API Rate Limits:** Bulk automated operations (such as mass triage evaluations) remain bounded by Groq Cloud API developer tier rate limits and rate-concurrency quotas.
- **Log Format Dependencies:** While standard audit formats (such as sample_audit.log) parse reliably, heavily customized or non-standard SIEM log exports require strict header alignment for optimal deterministic anomaly extraction.

7. Demonstration Index

YouTube Video Link: https://youtu.be/P0hejUG7d7c?si=na9R46uOnxGPTTjN

Feature ID & Name	Description	Video Timestamp
F01 - External Threat Advisory Ingestion	Fetches, parses, and stores live vulnerability records from CISA KEV and NIST NVD catalogs into the local SQLite database.	https://www.youtube.com/watch?v=P0hejUG7d7c&t=33s
F02 - Automated Threat Bulletin Summarization	Selects an advisory record and queries the Groq LPU inference engine to generate an executive summary rendered in rich HTML.	https://www.youtube.com/watch?v=P0hejUG7d7c&t=65s

F03 - Natural Language Threat Landscape Querying	Submits ad-hoc security questions to the AI chat interface / CLI query shell, utilizing session memory context.	https://www.youtube.com/watch?v=P0hejUG7d7c&t=87s
F04 - Insider Risk Log Correlation	Uploads an access log to parse behavioral outliers and cross-reference them against known CVE signatures.	https://www.youtube.com/watch?v=P0hejUG7d7c&t=121s
F05 - Automated Mitigation Strategy Generation & Execution	Formulates step-by-step remediation playbooks, executes BlockIpCommand actions, and tests undo rollback capabilities.	https://www.youtube.com/watch?v=P0hejUG7d7c&t=169s
F06 - Structured Indicator of Compromise (IoC) Extraction	Pastes unstructured threat text into the extraction panel to pull and normalize defanged IPs, hashes, and domains.	https://www.youtube.com/watch?v=P0hejUG7d7c&t=208s
F07 - Executive Threat Intelligence Report Export	Compiles single or multiple selected threat advisories into a structured Markdown briefing report saved locally.	https://www.youtube.com/watch?v=P0hejUG7d7c&t=258s
F08 - Historical Threat Repository Search	Performs fast, deterministic multi-criteria filtering across cached vulnerabilities (CVSS score thresholds) without consuming LLM API limits.	https://www.youtube.com/watch?v=P0hejUG7d7c&t=290s&pp=0gcJCWMAwfN6Pr3D
F09 - Dynamic Advisory Triage & Priority Scoring	Evaluates threat exploitability against local environment profiles to assign custom priority scores and justification notes.	https://www.youtube.com/watch?v=P0hejUG7d7c&t=322s
F10 - Security Control Gap Analysis	Cross-references active enterprise baseline security controls against vulnerability threat vectors to highlight coverage gaps.	https://www.youtube.com/watch?v=P0hejUG7d7c&t=358s
F11 - Multi-Select Batch Export	Holds Ctrl/Shift to select multiple CVE rows simultaneously and exports them into a consolidated batch intelligence briefing.	https://www.youtube.com/watch?v=P0hejUG7d7c&t=384s
F12 - Live Threat News Carousel	Live Threat News Carousel Footer	https://www.youtube.com/watch?v=P0hejUG7d7c&t=418s